Big Data Analytics
Prof. Dr. Martin Theobald
Summer Semester 2026 – University of Luxembourg
Exercise Sheet #1 – Due on March 26, 2026 (before the lecture starts)

Please upload the source code of your solutions on or before the due date to the provided Moodle assignment as a single zip file using your group id as the file name. Provide some brief instructions on how to run your solution to each problem in a file called `Problem_X.txt`*, and report also the most important results (such as number of results, runtimes, etc.) of your solutions to that problem within this file. Remember that all solutions may be submitted in groups of up to 3 students.*

New for 2026: Following the University Guidelines for the usage of Generative AI in teaching and learning, we explicitly encourage the usage of AI to solve this exercise sheet. Please indicate which AI platform you used and also summarize the most important prompting steps (if any) together with your solution to each problem. All solutions will be checked for completeness and their functionality!

Extended Word Counting in Apache Hadoop **12 Points**

Problem 1. Consider the three Java classes `HadoopWordCount.java`, `HadoopWordPairs.java` and `HadoopWordStripes.java` provided on Moodle to solve this exercise. Set up your local Hadoop environment by following the steps described in the "GettingStarted" guide. Note that in particular the solution to part (c) of this exercise is computationally quite costly and may be measured either by using your local Hadoop installation or via the IRIS HPC cluster (a similar "local" Hadoop installation on one of the IRIS nodes, e.g., in your home directory, is needed in this case).

(a) Modify the Java classes above, such that the `map` function emits only the following types of tokens to the reducers:

(1) *words* consisting only of sequences of lower-case letters `a-z`, including also the two dashes `_-`, with a total length of at least 6 characters and at most 24 characters per word; and

(2) *numbers* consisting only of sequences of digits `0-9`, including also the two decimal separators `.,` plus at most one leading dash `-`, with a total length of at least 4 characters and at most 16 characters per number.

That is, first turn all input strings (i.e., each line of a text file in Hadoop) into lower cases and then apply the above filters, e.g., via corresponding regular expressions. You may remove (i.e., split the input strings at) *any other non-word characters* between the words and numbers. See, e.g., `http://www.tutorialspoint.com/java/java_regular_expressions.htm` for a tutorial on using regular expressions in Java. **4 Points**

(b) Next, analyze the Wikipedia collection provided on Moodle as follows. Focus only on the `AA` subdirectory to solve the following points.

(1) Find all individual words (not numbers) with a total count of exactly 1,000.

(2) Find all word pairs (not numbers) with a total count of exactly 1,000 (using a distance of $m \leq 2$ for immediate neighbors of words in the mappers).

(3) Find the top-100 most frequent individual words (not numbers).

(4) Find the top-100 most frequent pairs of tokens, where the first token is a number and the second token is a word (again using a distance of $m \leq 2$ for immediate neighbors of numbers and words in the mappers).

Try to combine the above tasks (1)–(4) into as few MapReduce jobs as possible. Note that you may read the output files produced by one MapReduce job as input to another MapReduce job. No further processing outside the `map` and `reduce` functions is allowed (i.e., you in particular need to find a way to sort and filter the top-100 results directly in MapReduce)!

You may choose whether you prefer the `HadoopWordPairs.java` or the `HadoopWordStripes.java` example as basis for computing the frequencies of the pairs (one is sufficient). **6 Points**

(c) Continue your analysis step-by-step, including also the remaining `AB`, `AC`, ..., `AK` subdirectories of the provided collection of 41,784 Wikipedia articles. That is, include one more subdirectory at each step

and measure the runtime of your MapReduce jobs until completion. Report your detailed hardware setup and the runtimes in your `Problem_1.txt` file. Which conclusions can you draw with respect to the scalability of your solution? **2 Points**

Extended Word Counting in Apache Spark **12 Points**

Problem 2. Repeat the above steps in Apache Spark based on the `SparkWordCount.scala` example provided on Moodle[1]. You may directly extend the provided tokenizer to also extract immediate pairs of tokens (again with a distance of $m \leq 2$), and you may use any of the RDD *transformations* and *actions* provided in the lecture slides. Also here, try to minimize the amount of Spark transformations you need in order to complete the given tasks. Consider using the IRIS HPC cluster with the provided launcher scripts in particular for the larger jobs. Report your detailed hardware setup and the runtimes in your `Problem_2.txt` file.

Using DataFrames and Spark SQL **12 Points**

Problem 3. Consider the `DataFrame.scala` or `DataFrame.ipynb` scripts together with the `titanic.csv` dataset provided on Moodle to solve this problem.

(a) Create a user-defined function (UDF) called `MedianAge` that computes the *median age* over all passengers in the initial Spark DataFrame. Your function should automatically replace all `null` entries for the passengers' ages with the default integer value "23" while calculating the median age. Refer to the "User-Defined Aggregate Functions (UDAFs)" section in the Spark SQL programming guidelines[2] for an example. **4 Points**

(b) Implement the following query using only built-in DataFrame and/or SQL operations. Refer to the Spark SQL documentation[3] for the list of operations that can be performed on a DataFrame.

Query: *"Select the name and fare price of all passengers whose age is greater than the median age of all passengers who embarked on the Titanic."* **4 Points**

(c) Assume we wish to investigate the following (statistical) hypothesis:

Hypothesis: *"The average fare price of passengers who survived the sinking of the Titanic is not very different from (i.e., neither much below nor much above) the average fare price of passengers who did not survive the sinking of the Titanic."*

How would you verify the above hypothesis? Please provide an implementation based on Spark's built-in DataFrame and/or SQL operations. You may but you do not have to use a *statistical test* (such as the "Unequal Variance T-Test") to either accept or reject this hypothesis; a simple numeric comparison will also be sufficient. Summarize your results in your `Problem_3.txt` file. **4 Points**

[1] Alternatively, you may also consider the corresponding shell script `WordCount.scala` if you prefer using the interactive Spark shell or the Jupyter notebook `WordCount.ipynb` if you prefer using PySpark (both are also available from Moodle).
[2] `https://spark.apache.org/docs/latest/sql-ref-functions-udf-aggregate.html`
[3] `https://spark.apache.org/docs/latest/sql-programming-guide.html`